

MACHINE LEARNING SYSTEMS: PROJECT REPORT

Anonymous authors

Paper under double-blind review

ABSTRACT

This report investigates GPU-accelerated Top-K retrieval for large-scale similarity search. We implement exact and approximate pipelines using PyTorch and CuPy, integrating custom distance functions (L1, L2, Cosine, Dot Product) with batched computation. Our ANN approach combines KMeans clustering with local KNN refinement, while an IVFPQ baseline provides compressed retrieval via FAISS. We benchmark all methods across dimensions and dataset sizes, highlighting trade-offs between accuracy and speed. In the second part of the report, ML systems' supporting infrastructure is investigated, with a focus on queuing and batching techniques. We implement a custom batching and queuing algorithm designed to handle variable request loads. The resulting queue- and batch-enriched RAG system is tested on a dataset of artificially constructed queries under variable request loads using several configurations. We conclude that queuing and batching techniques improve the systems' performance under high load while maintaining or decreasing efficiency under low-to-medium loads. Finally, the overall ML system's infrastructure is considered, and conclusions are drawn on the dependencies between its components.

1 INFORMATION RETRIEVAL ON GPU (TASK 1)

1.1 MOTIVATION

Information Retrieval (IR) systems are central to modern machine learning, enabling fast access to relevant data in domains like web search, recommendation, and vector databases (Manning et al., 2008; Baeza-Yates & Ribeiro-Neto, 1999). These systems encode data as high-dimensional vectors and retrieve the most similar ones using metrics such as L1, L2, cosine, or dot product (Muja & Lowe, 2014).

To scale efficiently, IR pipelines often apply clustering (e.g., KMeans) to group similar data and reduce search space, a core strategy in ANN frameworks like FAISS and ScaNN (Djenouri et al., 2021; Douze et al., 2025; Guo et al., 2020). However, both clustering and similarity search on large datasets (e.g., $> 10^5$ vectors of $> 10^3$ dimensions) remain computationally intensive due to the scale of operations and memory bandwidth demands (Johnson et al., 2019; Li et al., 2023).

GPU acceleration addresses these challenges via massively parallel execution, memory-efficient batching, and high-throughput distance computation (Johnson et al., 2019; Guo et al., 2020). Libraries like PyTorch (Paszke et al., 2019), CuPy (Okuta et al., 2017), and FAISS provide GPU-optimised primitives for scalable IR development.

Such techniques are widely deployed in production systems like Milvus, Pinecone, and Facebook's Faiss stack, supporting real-time vector search at an industrial scale (Johnson et al., 2019; Pan et al., 2024).

1.2 OVERVIEW

We implemented a GPU-accelerated system for scalable similarity search and clustering on high-dimensional vector data. The system supports both exact and approximate retrieval via modular components built in PyTorch and CuPy, enabling fast distance computation, clustering, and Top-K search.

The pipeline begins with an input query embedded as a high-dimensional vector. This vector is passed to the GPU, where batched distance functions compute similarity scores against dataset vectors. The system then proceeds through one of two retrieval paths: (1) exact Top-K retrieval over the full dataset, or (2) approximate retrieval that uses KMeans clustering to select the closest cluster, followed by local KNN search.

Figure 1 illustrates the complete workflow, including the distance function module, retrieval branches, and final Top-K output. Both paths return the top-ranked results along with their associated metadata.

Our design emphasises three key principles: modularity, which allows components like distance functions and clustering to be interchanged; parallelism, to exploit GPU hardware for scalability; and memory efficiency, through batched data movement and computation. These align with strategies used in large-scale IR systems like FAISS and ScaNN (Johnson et al., 2019; Guo et al., 2020; Research, 2024).

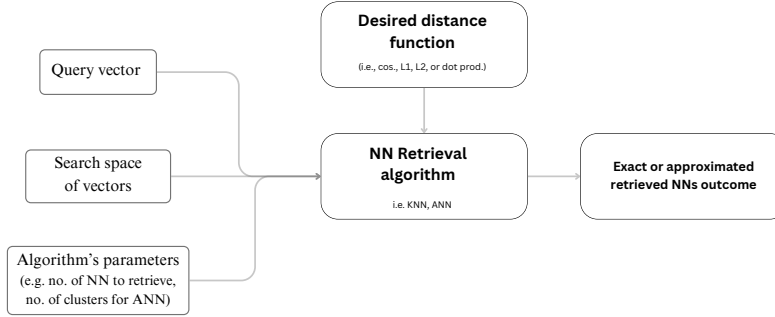


Figure 1: Workflow of our GPU-accelerated similarity search pipeline. Queries are processed using batched distance functions and passed through exact or approximate retrieval.

1.3 DISTANCE FUNCTIONS

Distance functions are central to similarity-based retrieval and clustering, providing a measure of vector dissimilarity crucial in algorithms like KMeans, KNN, and Top-K search (Baeza-Yates & Ribeiro-Neto, 1999). Their impact on system performance and accuracy depends on the dataset and application domain (Kumar et al., 2014).

We implemented four standard distance functions: Cosine, L2 (Euclidean), L1 (Manhattan), and Dot Product—using both CuPy and PyTorch. A multidimensional PyTorch version was then developed to support GPU-accelerated batched operations via broadcasting, norms, and other matrix ops. Full code is in Appendix C. Notably, we use the negative dot product for distance (since higher values imply proximity) and add $\epsilon = 10^{-8}$ in denominators to avoid division by zero.

We benchmarked all distance types across CPU, CuPy-GPU, and PyTorch-GPU backends for dimensions ranging from $D = 2$ to $D = 1,048,576$. As expected, CPU performs better at low D , but GPU speedups grow significantly at higher dimensions. PyTorch reached a maximum speedup of $11.93\times$ over CPU at $D = 2^{20}$ ¹. Figure 3 (Appendix B) visualises this trend.

Cosine and dot product were fastest at low D (e.g., $D < 1024$), while L1 dominated in high-dimensional or sparse settings. These findings informed our distance choice for Top-K (Section 1.4), KMeans (Appendix E), and ANN (Section 1.5). Our benchmarks highlight the importance of choosing distance metrics based on application and dimensionality. GPU acceleration, batching, and precision-aware implementation proved essential for scalability. These multidimensional distance functions power downstream modules like KMeans and KNN, enabling efficient batched comparisons and explaining performance trends as D increases (e.g., $D = 2$ to 2^{20}). This supports the clustering speedups observed, despite CPU comparisons being limited to standalone kernels.

¹See Appendix A for complete benchmarking tables.

1.4 TOP-K RETRIEVAL

Top-K retrieval returns the K most relevant dataset vectors for a given query by computing all pairwise distances ($O(ND)$) and partially sorting them ($O(N \log K)$) (Muja & Lowe, 2014). While accurate, this becomes expensive at scale. To address this, we implemented two retrieval pipelines: (1) a custom ANN method based on KMeans+KNN and (2) an IVFPQ-based approach using FAISS (see Section 1.5). Both reduce runtime but differ in memory usage, compression, and recall.

In the exact pipeline, we compute distances using batched GPU kernels (Cosine, L2, L1, Dot Product), followed by `torch.topk()`, which uses CUDA-optimised radix/heap selection (Paszke et al., 2019). To avoid out-of-memory (OOM) errors, we use chunking (100K splits) with float32 precision. `float16` caused instability and invalid indices, while large-scale datasets ($N > 10^5$) required batching and vectorised ops to support all-to-all comparisons—used in both retrieval and clustering.

Our scalable KMeans+KNN pipeline partitions the dataset into k clusters using our own GPU-based KMeans, then assigns each query to its nearest centroid and performs KNN within that cluster. This reduces complexity from $O(N)$ to $O(N/k)$ with minimal recall loss. Both ANN methods support all distance types and share a modular, GPU-accelerated framework. Implementations are in Appendix F, Listings 1–3.

We benchmarked all pipelines² on datasets up to $N = 200,000$ and $D = 2^{20}$. Our ANN approach consistently achieved $> 10\times$ speedup with $\text{recall@1000} > 0.91$. At $D = 1,048,576$, PyTorch-GPU achieved a peak $11.93\times$ speedup over CPU. Using $k = 64$ clusters and single-probe yielded recall $\approx 70\%$, balancing efficiency and quality.

These results align with recent GPU-accelerated ANN research (Malkov & Yashunin, 2020; V. et al., 2024), highlighting trade-offs between speed and recall. Appendix D (Figure 4) shows near-linear KNN scaling with N at fixed $D = 100$. GPU vs. CPU comparisons for KMeans in Appendix E (Figure 5) show strong acceleration, especially for L1 and Dot Product at $D = 1024$.

KMeans and KNN runtime trends follow the same distance functions profiled in Section 1.3, where GPU speedups increase beyond $D > 2^{10}$. Both modules leverage batched GPU operations: KNN processes candidates in query-aligned bulks, while KMeans compares all points to centroids per iteration—enabling efficient large-scale retrieval.

1.5 ADVANCED OPTIMISATION

To reduce latency and memory usage on large-scale datasets, we implemented an approximate nearest neighbour (ANN) pipeline using Inverted File Indexing with Product Quantisation (IVFPQ) via FAISS (Johnson et al., 2019). IVFPQ compresses high-dimensional vectors and enables fast search by combining (1) coarse quantization—partitioning the dataset into K_{IVF} clusters using KMeans—and (2) vector compression via Product Quantisation (PQ), where each vector is encoded into low-bit subspaces (Jégou et al., 2011).

At query time, the K_{probe} nearest clusters are searched using precomputed PQ lookup tables. We tested FAISS-GPU configurations with $K_{\text{IVF}} \in \{32, 64\}$, $K_{\text{probe}} \in \{4, 16\}$, and PQ codebooks $K_{\text{PQ}} \in \{5, 10\}$. On a dataset with $N = 200,000$, $D = 100$, IVFPQ delivered over $5\times$ speedup compared to exact Top-K, with recall ranging from 0.07 to 0.29. Full results appear in Appendix G, Tables 4–6, and Figure 6 compares recall and runtime across methods.

The best recall was achieved with $K_{\text{IVF}} = 64$ and $K_{\text{probe}} = 16$, confirming the benefits of multi-cluster probing. However, even the top IVFPQ setup underperformed our custom KMeans+KNN pipeline (Section 1.4), which reached up to 0.81 recall with higher memory usage.

This reflects IVFPQ’s design trade-off: favouring speed and scalability in latency-sensitive applications (e.g., web search, recommender systems) (Jégou et al., 2011; Johnson et al., 2019; Aumüller et al., 2019), while KMeans+KNN offers a better speed–accuracy balance for high-recall scenarios (e.g., legal or biomedical domains).

²Metrics in Section 1.3 and Appendix A.

A final comparison of all three pipelines—Exact, KMeans+KNN, and IVFPQ—appears in Appendix G, Table 6. Performance plots (Figure 6) illustrate recall–latency trade-offs. Implementation details for IVFPQ are provided in Appendix H.

2 MODEL SERVING SYSTEMS (TASK 2)

2.1 MOTIVATION

Designing a high quality machine learning system requires more than training accurate and efficient models – it demands a robust serving infrastructure capable of handling requests at scale. Online inference introduces several challenges that, if unaddressed, may lead to severe performance degradation.

Such systems are faced with a challenge of processing a stream of unpredictable, user-generated requests, often being under tight constraints of reducing overall system latency. Without proper batching or queuing techniques online inference systems may process user requests individually, one by one. This results in a compounding effect of several factors: high per-request overhead caused by communication between workers, numerous model invocations, underutilisation of GPU/TPU resources, and susceptibility to traffic spikes at busy times.

Request queuing places the incoming requests into a controlled buffer – a temporary storage area in memory that is used to hold data during transfer between two systems (Patke et al., 2024). This smooths the spikes in traffic, gives more control over the scheduling operations and prevents system overload. **Batching**, on the other hand, aggregates a certain number of requests into a group, which can then be sent to the model for inference at once. This allows the system to fully leverage parallel processing capabilities of hardware accelerators (GPU/TPU) and is ideal for vectorised operations common in machine learning (such as matrix-matrix multiplication)(Crankshaw et al., 2017).

Several production-grade ML serving frameworks employ batching and queuing techniques. TensorFlow Serving supports flexible batching with a configurable batch size and timeout (Abadi et al., 2015). BentoML is an open-source project supporting adaptive batching which adapts to various-load requests (Yang et al., 2024). Ray Serve is a scalable ML model serving framework that leverages asynchronous request queuing (Moritz et al., 2025).

2.2 OVERVIEW

We implemented a FastAPI-based (Ramírez, 2024) Retrieval Augmented Generation (RAG) pipeline that handles inference requests via an asynchronous queuing and batch processing approach. Incoming queries are placed into a thread-safe FIFO (First-In-First-Out) queue, and a dedicated background worker thread retrieves and aggregates them into batches of pre-defined size at configurable intervals. Each batch then gets processed through a RAG pipeline, which embeds the queries, employs a top-k nearest neighbours search function – in one experiment leveraging the KNN algorithm developed for Task 1 – and generates responses using Qwen2.5-1.5B-Instruct LLM (Team, 2024).

This design allows our system to balance the GPU throughput and latency while maintaining performance under fluctuating user request loads. The system workflow can be summarised as follows:

1. Incoming requests arrive asynchronously and are placed in a FIFO queue.
2. A background thread continuously monitors the queue, retrieving requests until **either** a number of collected requests reaches **or** The waiting time reaches `MAX_WAITING_TIME`
3. Once either of the conditions is satisfied, a batch is formed. If the number of requests exceeds `MAX_BATCH_SIZE`, only the earliest requests are taken.
4. Each batch of queries then gets passed onto the RAG pipeline:
 - (a) The system computes query embeddings.
 - (b) Top-k nearest search function is applied retrieving the k closest documents based on a selected similarity metric.
 - (c) A prompt is constructed using provided queries and retrieved context.
 - (d) Finally, a response is generated using a text generation pipeline.

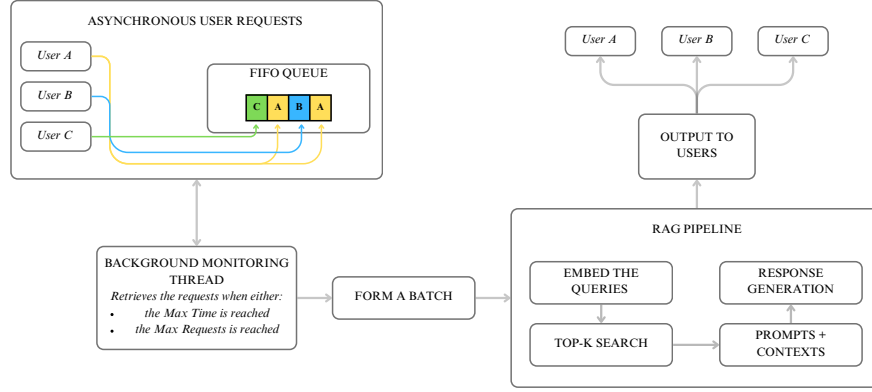


Figure 2: A workflow figure depicting the RAG pipeline with request queuing and batching. The embedding model used is **intfloat/multilingual-e5-large-instruct**, the Top-K function is based on Task 1 implementation, and the LLM used is **Qwen2.5-1.5B-Instruct**.

5. Responses are asynchronously returned to users via the `Future()` instance (Foundation, 2024)

Figure 2 shows a diagram summarising and visualising the process outlined above.

2.3 SYSTEM MEASUREMENT

To evaluate the final system, we constructed a synthetic dataset of 30 queries using OpenAI’s ChatGPT-4o model (OpenAI, 2024) and employed a Python load testing script to send them at rates ranging from 0.1 to 100 requests/s. The script launched a thread for every query and recorded the time taken via Python’s native `time.time()` function (Python Software Foundation, 2024b). This allowed us to simulate concurrent users and various load conditions.

We measured average inference time per query with a variable Requests Per Second (RPS) rate, as it allowed us to assess the system’s latency and responsiveness – two crucial components of an online inference system. Additionally, varying the request rates simulated the system performance under real-world load conditions and allowed to see the gains from queuing and batching. Inference time was computed directly in the script and printed out along with the query and LLM’s truncated answer. To ensure fair testing, each rate run was performed with identical parameters (unless specified otherwise) and on the same set of queries. The results were compared across the following configurations:

- Baseline (no queuing or batching) RAG pipeline with and without our `knn` implementation.
- Full pipeline including batching and queuing with a varying number of `MAX_BATCH_SIZE` and `MAX_WAITING_TIME` parameters.
- Different hardware: **NVIDIA TITAN X PASCAL** and **NVIDIA GeForce GTX 1060 6GB**
- Two large language models: A heavy **Qwen2.5-1.5B-Instruct** and a lighter **facebook/opt-125m** (Team, 2024)(Zhang et al., 2022)

2.4 REQUEST QUEUE AND BATCHER

Request queuer and batcher are the foundational techniques used in ML systems’ supporting infrastructure. They help ensure system stability under the variable load of asynchronous user requests, prevent failures during peak times and help fully utilize hardware accelerators’ power parallelism capabilities.

We implemented the request queue using Python’s built-in `queue.Queue()` data structure (Python Software Foundation, 2024a) which gathers incoming user requests asynchronously, al-

lowing the system to retrieve them in a controlled manner. A separate background thread hosts the batcher, which continuously monitors the request queue. It triggers the RAG pipeline when either of the following conditions is satisfied: a) the batch size reached the maximum batch size or b) the maximum wait time for retrieving the requests was reached. Finally, to return the results asynchronously, the `concurrent.futures.Future()` (Foundation, 2024) object was used. This allowed the main thread to wait and gather inference results from the corresponding worker efficiently.

Table 7 presents the results of our experimental evaluation. All the systems, except for the one annotated "light" used larger, Qwen2.5B LLM on TTITAN X Pascal. The lighter system used facebook's 125M parameter LLM on NVIDIA GeForce GTX 1060 6GB. As expected, the RAG-only pipeline executed on the CPU showed the weakest performance due to its inability to exploit parallelism. A baseline RAG pipeline without queuing or batching performed adequately under low request rates, however, experienced a noticeable performance drop as the RPS increased. This highlights the limitations of unoptimised ML serving pipelines under high-stress conditions. The lightweight configuration had the lowest response rates out of all the setups, however, exhibited high hallucination rates and struggled with high RPS – demonstrating the trade-off between high performance and accuracy. The RAG pipeline (without queue or batcher) using the KNN function from task 1 outperformed the baseline across most RPS (especially the higher ones) suggesting the efficiency of our implementation.

Adding the queue and batcher significantly improved performance under high load despite introducing some overhead at lower RPS. This could be attributed to additional thread management and latency associated with the waiting time to form batches. This indicates the batching and queuing mechanism is viable for dealing with variable loads and enhances the ML systems' performance in real-world scenarios. Finally, we evaluated the effect of tuning batcher parameters. Comparing the results of the system with `batch_size` of 16 vs 4, and `wait_time` of 4s vs 2s, we observed the higher batch size often hurt the performance, especially under low to moderate RPS. We attribute this to our relatively small dataset, where larger batches added wait time without showing any computational gains.

3 DISCUSSION OF TASK SYNERGY

GPU kernel optimisation and the optimisation of the ML supporting infrastructure are seemingly modulated tasks; however, they have a direct impact on one another and need to be considered jointly. In a real-world ML pipeline, GPU kernels sit at the heart of inference and retrieval workloads; however, their efficiency is only realised when the upstream system (covered in Task 2) feeds them with well-formed, appropriately-sized batches of requests. Queuing and batching thus directly dictate how and when the GPU kernels get invoked, affecting hardware accelerators' utilisation, memory efficiency and throughput. On the other hand, having a poorly optimised GPU kernel, an efficient supporting infrastructure will not be able to provide a significant performance gain.

The two tasks are clearly correlated – the effectiveness of one stagnates with a poor optimisation of another. This introduces certain trade-offs. For instance, increasing batch size improves GPU throughput (Kosaian et al., 2021) but can also increase latency for returning individual requests due to the wait time in the queue. On the other hand, a GPU kernel optimised for large batches may not exhibit any performance gains or even show a degradation with smaller batch sizes (Kosaian et al., 2021).

Optimising GPU kernels without corresponding upstream systems' optimisation may result in bottlenecks that mask the increased efficiency. For example, the kernel optimisation may not be noticeable if the requests are improperly batched or the batching parameters are set inefficiently. Conversely, improving queuing and batching techniques can show a noticeable improvement even without corresponding kernel optimisations – by addressing the patterns of GPU invocation.

Finally, there are certain techniques that can be used to further enhance the system's efficiency. For example, adaptive batching (Cheng et al., 2018) ensures the batches are adjusted dynamically according to the current load and observed GPU throughput. Additionally, fair scheduling (Zhang et al., 2017) or back-end fetch metrics (Merkel, 2025) can be employed in the queuing algorithms to balance the users' priorities and deal with variable-sized request rates.

REFERENCES

- Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, et al. Tensorflow: Large-scale machine learning on heterogeneous systems / tensorflow serving, 2015. URL https://www.tensorflow.org/tfx/serving/serving_config#batching_configuration. Software available from tensorflow.org.
- Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. Ann-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems*, 87:101374, 2019. doi: 10.1016/j.is.2019.02.006. URL <https://www.sciencedirect.com/science/article/pii/S0306437918303685>.
- Ricardo Baeza-Yates and Berthier Ribeiro-Neto. *Modern Information Retrieval*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, 1999. URL <https://web.cs.ucla.edu/~miodrag/cs259-security/baeza-yates99modern.pdf>.
- Dazhao Cheng, Xiaobo Zhou, Yu Wang, and Changjun Jiang. Adaptive scheduling of parallel jobs with dynamic batching in spark streaming. *IEEE Transactions on Parallel and Distributed Systems*, 29(12):2672–2685, 2018. doi: 10.1109/TPDS.2018.2846234.
- Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. Clipper: A Low-Latency online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pp. 613–627, Boston, MA, March 2017. USENIX Association. ISBN 978-1-931971-37-9. URL <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/crankshaw>.
- Youcef Djenouri, Asma Belhadi, Djamel Djenouri, and Jerry Chun-Wei Lin. Cluster-based information retrieval using pattern mining. *Applied Intelligence*, 51:1888–1903, 2021. doi: 10.1007/s10489-020-01922-x. URL <https://doi.org/10.1007/s10489-020-01922-x>.
- Matthijs Douze, Jeff Johnson, Maria Lomeli, Alexandr Guzhva, Gergely Szilvasy, Lucas Hosseini, Chengqi Deng, Pierre-Emmanuel Mazaré, and Hervé Jégou. The faiss library. *arXiv preprint arXiv:2401.08281*, 2025. URL <https://arxiv.org/abs/2401.08281>.
- Python Software Foundation. *concurrent.futures — Launching parallel tasks*, 2024. URL <https://docs.python.org/3/library/concurrent.futures.html>. Python 3.12 documentation.
- Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*, 2020. URL <https://proceedings.mlr.press/v119/guo20h/guo20h.pdf>.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 2019. URL <https://arxiv.org/abs/1702.08734>.
- Herve Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011. doi: 10.1109/TPAMI.2010.57.
- Jack Kosaian, Amar Phanishayee, Matthai Philipose, Debadeepta Dey, and Rashmi Vinayak. Boosting the throughput and accelerator utilization of specialized cnn inference beyond increasing batch size. In *Proceedings of the 38th International Conference on Machine Learning (ICML 2021)*, volume 139 of *Proceedings of Machine Learning Research*, pp. 5731–5741. PMLR, 2021. URL <https://proceedings.mlr.press/v139/kosaian21a.html>. Accessed April 2025.
- Vipin Kumar, J. K. Chhabra, and Dinesh Kumar. Impact of distance measures on the performance of clustering algorithms. In *Proceedings of the International Conference on Recent Advances in Intelligent Informatics*, pp. 325–335. Springer, 2014. doi: 10.1007/978-81-322-1665-0_17. URL https://link.springer.com/chapter/10.1007/978-81-322-1665-0_17.

- Mi Li, Eibe Frank, and Bernhard Pfahringer. Large-scale k-means clustering using gpus. *Data Mining and Knowledge Discovery*, 37(1):67–109, 2023. doi: 10.1007/s10618-022-00869-6. URL <https://link.springer.com/article/10.1007/s10618-022-00869-6>.
- Yu A. Malkov and D. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. In *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2020. URL <https://arxiv.org/abs/1603.09320>.
- Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008. URL <https://nlp.stanford.edu/IR-book/>.
- Benjamin Merkel. Efficient request queueing – optimizing llm performance. *Hugging Face blog*, 2025. URL <https://huggingface.co/blog/tngtech/llm-performance-request-queueing>. Community article.
- Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray serve: Scalable and programmable serving, 2025. URL <https://docs.ray.io/en/latest/serve/index.html>. Accessed April 2025.
- Marius Muja and David G. Lowe. Scalable nearest neighbor algorithms for high dimensional data. In *IEEE TPAMI*, volume 36, pp. 2227–2240, 2014. URL <https://ieeexplore.ieee.org/document/6809191>.
- Ryosuke Okuta, Yuya Unno, Daisuke Nishino, Shohei Hido, and Crissman Loomis. Cupy: A numpy-compatible library for nvidia gpu calculations. *Proceedings of Workshop on Machine Learning Systems (LearningSys) in NIPS*, 2017. URL http://learningsys.org/nips17/assets/papers/paper_16.pdf.
- OpenAI. *GPT-4o*, 2024. URL <https://openai.com/index/hello-gpt-4o/>.
- James Jie Pan, Jianguo Wang, and Guoliang Li. Survey of vector database management systems. *The VLDB Journal*, 33:1591–1615, 2024. doi: 10.1007/s00778-024-00864-x. URL <https://doi.org/10.1007/s00778-024-00864-x>.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://arxiv.org/abs/1912.01703>. NeurIPS 2019.
- Archit Patke, Dhmath Reddy, Saurabh Jha, Haoran Qiu, Christian Pinto, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. Queue management for slo-oriented large language model serving. *arXiv preprint arXiv:2407.00047*, 2024. URL <https://arxiv.org/abs/2407.00047>.
- Python Software Foundation. *asyncio.Queue*, 2024a. URL <https://docs.python.org/3/library/asyncio-queue.html>. Python 3.12 documentation.
- Python Software Foundation. *time.time*, 2024b. URL <https://docs.python.org/3/library/time.html>. Python 3.12 documentation.
- Sebastián Ramírez. Fastapi, 2024. URL <https://fastapi.tiangolo.com>. Accessed April 2025.
- Facebook AI Research. Faiss: A library for efficient similarity search. <https://github.com/facebookresearch/faiss>, 2024. Accessed March 2025.
- Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL <https://qwenlm.github.io/blog/qwen2.5/>.
- Karthik V., Saim Khan, Somesh Singh, Harsha Vardhan Simhadri, and Jyothi Vedurada. Bang: Billion-scale approximate nearest neighbor search using a single gpu. *arXiv preprint arXiv:2401.11324*, 2024. URL <https://arxiv.org/abs/2401.11324>.

Chaoyu Yang, Sean Sheng, Aaron Pham, Shenyang Zhao, Sauyon Lee, Bo Jiang, Fog Dong, Xipeng Guan, and Frost Ming. Bentoml: The framework for building scalable ai applications, 2024. URL <https://bentoml.com/>. Accessed April 2025.

Haoyu Zhang, Logan Stafman, Andrew Or, and Michael J. Freedman. Smaq: Quality-driven scheduling for distributed machine learning. In *Proceedings of the 2017 Symposium on Cloud Computing (SoCC '17)*, pp. 390–404, New York, NY, USA, 2017. Association for Computing Machinery. doi: 10.1145/3127479.3127490. URL <https://doi.org/10.1145/3127479.3127490>. Accessed April 2025.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. Opt: Open pre-trained transformer language models, 2022.

A TOP-K DISTANCE FUNCTION BENCHMARK RESULTS

This appendix presents detailed execution times for our Top-K retrieval pipeline using different distance functions and vector dimensionalities. All benchmarking results were collected using the GPU-accelerated Top-K implementation described in Section 1.4, which computes pairwise distances and applies partial sorting via `torch.topk()`. Four distance functions—Cosine, L2, L1, and Dot Product—were tested across CPU, CuPy-GPU, and PyTorch-GPU backends. These results are also used to assess the runtime performance of the distance functions themselves (as discussed in Section 1.1), making this the definitive runtime benchmark for both Top-K search and distance function comparisons.

Dimension (D)	Distance Function	CPU (μ s)	CuPy-GPU (μ s)	PyTorch-GPU (μ s)
Performance for D = 2				
2	Cosine	95.70	515.20 (0.19 $\times$)	186.74 (0.51 $\times$)
2	L2	23.94	177.12 (0.14 $\times$)	65.99 (0.36 $\times$)
2	Dot	7.82	153.06 (0.05 $\times$)	35.72 (0.22 $\times$)
2	L1	16.53	94.67 (0.17 $\times$)	50.00 (0.33 $\times$)
Performance for D = 32				
32	Cosine	75.80	518.58 (0.15 $\times$)	171.12 (0.44 $\times$)
32	L2	23.63	176.03 (0.13 $\times$)	66.83 (0.35 $\times$)
32	Dot	7.71	149.36 (0.05 $\times$)	33.69 (0.23 $\times$)
32	L1	16.52	93.42 (0.18 $\times$)	50.42 (0.33 $\times$)
Performance for D = 1024				
1024	Cosine	78.95	512.10 (0.15 $\times$)	177.57 (0.44 $\times$)
1024	L2	25.14	179.29 (0.14 $\times$)	65.80 (0.38 $\times$)
1024	Dot	7.85	149.94 (0.05 $\times$)	33.04 (0.24 $\times$)
1024	L1	17.55	96.55 (0.18 $\times$)	49.83 (0.35 $\times$)
Performance for D = 32,768				
32768	Cosine	119.92	576.16 (0.21 $\times$)	182.78 (0.66 $\times$)
32768	L2	69.79	191.48 (0.36 $\times$)	67.75 (1.03 $\times$)
32768	Dot	20.77	163.17 (0.13 $\times$)	34.17 (0.61 $\times$)
32768	L1	62.17	111.49 (0.56 $\times$)	51.99 (1.20 $\times$)
Performance for D = 1,048,576				
1048576	Cosine	1313.29	553.65 (2.37 $\times$)	193.31 (6.79 $\times$)
1048576	L2	2021.67	1402.84 (1.44 $\times$)	178.46 (11.33 $\times$)
1048576	Dot	386.74	216.36 (1.79 $\times$)	60.05 (6.44 $\times$)
1048576	L1	2105.29	319.49 (6.59 $\times$)	176.45 (11.93 $\times$)

Table 1: Full benchmark results for all distance functions across increasing vector dimensions. Speedup values in parentheses represent performance relative to CPU.

B EXECUTION TIME ANALYSIS FOR L1 DISTANCE

Figure 3 illustrates execution time trends for L1 distance across all platforms. PyTorch-GPU surpasses CPU performance beyond $D = 10^3$, showcasing the benefits of GPU acceleration for high-dimensional IR tasks.

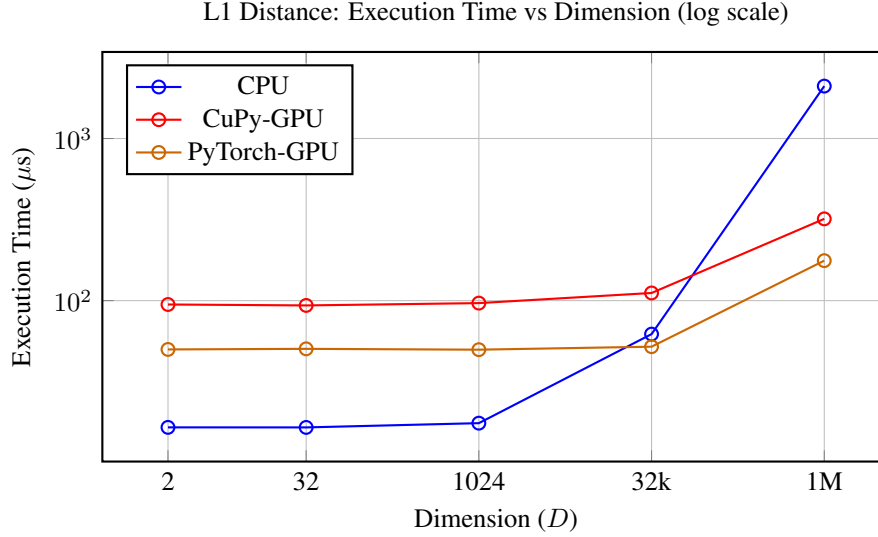


Figure 3: Execution time of L1 distance on CPU, CuPy-GPU, and PyTorch-GPU across increasing dimensionality. Log-scaled to highlight exponential growth and GPU speedups.

C DISTANCE FUNCTION IMPLEMENTATIONS

We provide below the PyTorch implementations of the four distance functions used throughout our system. Each is optimized for batched execution and reused in Top-K, KMeans, and ANN.

```
# -----
# 1.1 - Distance-computing functions
# -----
# Each distance formula is implemented through torch (default), cupy and
# torch for multi-dimentional vectors.

# Cosine distance:  $d(X, Y) = 1 - (X \cdot Y) / (|X| |Y|)$ 
# (an eps of 1e-8 is added to the denominator to avoid division by zero)
def cos_dist(X:torch.Tensor, Y:torch.Tensor) -> torch.Tensor:
    """
    INPUT
    - X[D],Y[D]: two vectors of the same dimension D
    OUTPUT
    - distance: the cosine distance between X and Y
    """
    return 1 - (torch.dot(X, Y) / (torch.norm(X) * torch.norm(Y) + 1e-8))
def cos_dist_cupy(X:cp.ndarray, Y:cp.ndarray) -> cp.ndarray:
    """
    INPUT
    - X[D],Y[D]: two vectors of the same dimension D
    OUTPUT
    - distance: the cosine distance between X and Y
    """
    return 1 - (cp.dot(X, Y) / (cp.linalg.norm(X) * cp.linalg.norm(Y) + 1e-8))
def cos_dist_multidim(X:torch.Tensor, Y:torch.Tensor) -> torch.Tensor:
    """
    INPUT
    - X,Y: two multidimensional vectors in the same space
    OUTPUT
    - distance: the cosine distance between X and Y
    """
    return 1 - (torch.sum(X * Y, dim=-1) / (torch.norm(X, dim=-1) * torch.
        norm(Y, dim=-1) + 1e-8))

# L2 (Euclidean) distance:  $d(X, Y) = \sqrt{\sum (X_i - Y_i)^2}$ 
def L2_dist(X:torch.Tensor, Y:torch.Tensor) -> torch.Tensor:
    """
    INPUT
    - X[D],Y[D]: two vectors of the same dimension D
    OUTPUT
    - distance: the Euclidean (L2) distance between X and Y
    """
    return torch.sqrt(torch.sum((X - Y) ** 2))
def L2_dist_cupy(X:cp.ndarray, Y:cp.ndarray) -> cp.ndarray:
    """
    INPUT
    - X[D],Y[D]: two vectors of the same dimension D
    OUTPUT
    - distance: the Euclidean (L2) distance between X and Y
    """
    return cp.sqrt(cp.sum((X - Y) ** 2))
def L2_dist_multidim(X:torch.Tensor, Y:torch.Tensor) -> torch.Tensor:
    """
    INPUT
    - X,Y: two multidimensional vectors in the same space
    OUTPUT
    - distance: the Euclidean (L2) distance between X and Y
    """
    return torch.norm(X - Y, dim=-1)
```

```

648
649 # Dot-product distance:  $d(X, Y) = -(X \times Y)$ 
650 # (negative dot product is used, since high  $X \times Y$  values mean vectors'
651   proximity)
652 def dot_dist(X:torch.Tensor, Y:torch.Tensor) -> torch.Tensor:
653     """
654     INPUT
655     - X[D],Y[D]: two vectors of the same dimension D
656     OUTPUT
657     - distance: the dot-product distance between X and Y
658     """
659     return - torch.dot(X, Y)
660 def dot_dist_cupy(X:cp.ndarray, Y:cp.ndarray) -> cp.ndarray:
661     """
662     INPUT
663     - X[D],Y[D]: two vectors of the same dimension D
664     OUTPUT
665     - distance: the dot-product distance between X and Y
666     """
667     return - cp.dot(X, Y)
668 def dot_dist_multidim(X:torch.Tensor, Y:torch.Tensor) -> torch.Tensor:
669     """
670     INPUT
671     - X,Y: two multidimensional vectors in the same space
672     OUTPUT
673     - distance: the dot-product distance between X and Y
674     """
675     return - torch.sum(X * Y, dim=-1)
676
677 # L1 (Manhattan) distance:  $d(X, Y) = \sum(|X_i - Y_i|)$ 
678 def L1_dist(X:torch.Tensor, Y:torch.Tensor) -> torch.Tensor:
679     """
680     INPUT
681     - X[D],Y[D]: two vectors of the same dimension D
682     OUTPUT
683     - distance: the Manhattan (L1) distance between X and Y
684     """
685     return torch.sum(torch.abs(X - Y))
686 def L1_dist_cupy(X:cp.ndarray, Y:cp.ndarray) -> cp.ndarray:
687     """
688     INPUT
689     - X[D],Y[D]: two vectors of the same dimension D
690     OUTPUT
691     - distance: the Manhattan (L1) distance between X and Y
692     """
693     return cp.sum(cp.abs(X - Y))
694 def L1_dist_multidim(X:torch.Tensor, Y:torch.Tensor) -> torch.Tensor:
695     """
696     INPUT
697     - X,Y: two multidimensional vectors in the same space
698     OUTPUT
699     - distance: the Manhattan (L1) distance between X and Y
700     """
701     return torch.sum(torch.abs(X - Y), dim=-1)
702
703 # Mapping functions for later usage
704 distance_types = ['cos', 'L2', 'dot', 'L1']
705 dist_functions = {'cos':cos_dist, 'L2':L2_dist, 'dot':dot_dist, 'L1':
706   L1_dist}
707 dist_multidim_functions = {'cos':cos_dist_multidim, 'L2':L2_dist_multidim
708   , 'dot':dot_dist_multidim, 'L1':L1_dist_multidim}

```


D KNN RESULTS ACROSS DATASET SIZE

Here we report the runtime for KNN search with dimensionality fixed at $D = 100$, while varying dataset size N . As seen in Table 2, runtime scales approximately linearly with N .

Dataset Size (N)	Cosine	L2	Dot Product	L1
1K	2.83 ms	0.56 ms	0.67 ms	0.60 ms
4K	1.18 ms	0.99 ms	1.03 ms	1.03 ms
4M	1902.27 ms	1927.41 ms	1905.19 ms	1918.43 ms

Table 2: KNN execution time at $D = 100$ for increasing dataset size N .

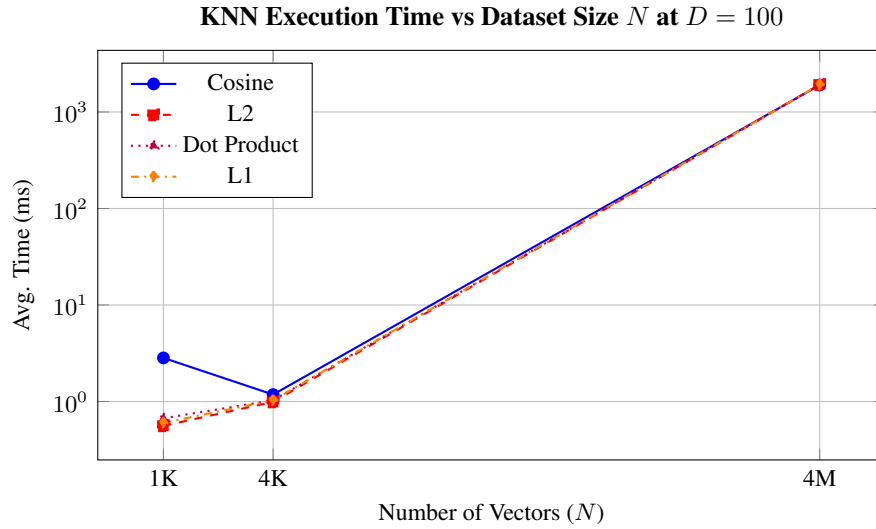


Figure 4: Execution time of KNN for different distance functions across increasing dataset sizes N at $D = 100$.

E KMEANS GPU VS CPU PERFORMANCE

This section presents the benchmark results of KMeans clustering at different dimensions on CPU vs GPU. Execution time and speedup values are shown in Table 3.

D	Distance Function	CPU (ms)	GPU (ms)
2	Cosine	26.41	26.89 (0.98×)
2	L2	20.21	27.27 (0.74×)
2	Dot Product	867.41	1008.82 (0.86×)
2	L1	24.47	25.86 (0.94×)
1024	Cosine	210.62	7.41 (28.43×)
1024	L2	268.26	10.01 (26.81×)
1024	Dot Product	198.36	7.17 (27.67×)
1024	L1	831.54	18.55 (44.83×)

Table 3: Execution time comparison of CPU vs. GPU for KMeans at $D = 2$ and $D = 1024$.

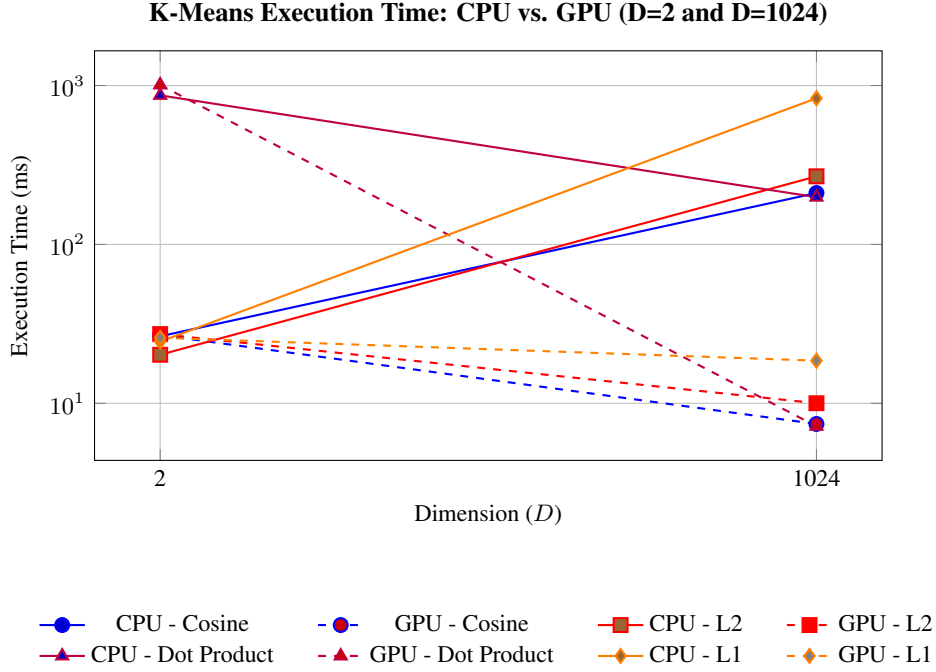


Figure 5: Execution time of K-means on CPU vs. GPU across different distance functions at $D = 2$ and $D = 1024$ (log scale). This figure highlights the significant speedups achieved by GPU execution at higher dimensionality, particularly for L1 and Dot Product distances.

F TOP-K AND ANN IMPLEMENTATION DETAILS

This appendix presents the complete implementation code for our Top-K, KMeans, and KMeans+KNN-based ANN pipelines, all implemented using GPU-accelerated PyTorch.

LISTING 1: TOP-K NEAREST NEIGHBOR SEARCH

Listing 1: Top-K nearest neighbors (KNN) using batched GPU distance functions

```
def our_knn(N:int, D:int, A:torch.Tensor, X:torch.Tensor, K:int,
           distance_metric:str='dot', batch_size:int=100000):
    if K > N: raise ValueError("Invalid K, larger than N")
    if distance_metric not in distance_types: raise ValueError(f"Invalid
        distance metric: {distance_metric}.")
    dist_multidim_func = dist_multidim_functions[distance_metric]
    A = torch.as_tensor(A, dtype=DTYPE, device='cuda')
    device = A.device
    X = torch.as_tensor(X, dtype=DTYPE, device=device)

    num_batches = (N + batch_size - 1) // batch_size
    top_k_distances = torch.empty(K, dtype=DTYPE, device=device)
    top_k_indices = torch.empty(K, dtype=torch.long, device=device)

    for i in range(num_batches):
        batch_start = i * batch_size
        batch_end = min((i + 1) * batch_size, N)
        batch = A[batch_start:batch_end]
        distances = dist_multidim_func(batch, X)
        topk_in_batch = min(K, distances.size(0))
        batch_top_k_distances, batch_top_k_indices = torch.topk(distances,
            topk_in_batch, largest=False)
        batch_top_k_indices += batch_start

        if i == 0:
            top_k_distances = batch_top_k_distances
            top_k_indices = batch_top_k_indices
        else:
            temp_distances = torch.cat((top_k_distances,
                batch_top_k_distances))
            temp_indices = torch.cat((top_k_indices, batch_top_k_indices))
            topk_in_merged = min(K, temp_distances.size(0))
            top_k_distances, indices = torch.topk(temp_distances,
                topk_in_merged, largest=False)
            top_k_indices = temp_indices[indices]

    return top_k_indices, top_k_distances
```

LISTING 2: KMEANS CLUSTERING ON GPU

Listing 2: GPU-based KMeans clustering with distance metric selection

```
def our_kmeans(N:int, D:int, A:torch.Tensor, K:int, distance_metric:str='
    dot', max_iters:int=1000, tol:float=1e-4, device:str='cuda'):
    if distance_metric not in distance_types: raise ValueError(f"Invalid
        distance metric: {distance_metric}.")
    dist_multidim_func = dist_multidim_functions[distance_metric]
    A = torch.as_tensor(A, dtype=DTYPE, device=device)
    centroids = A[torch.randperm(N)[:K]]

    for iteration in range(max_iters):
        prev_centroids = centroids.clone()
        distances = dist_multidim_func(A[:, None, :], centroids[None, :,
            :])
        clusters = torch.argmin(distances, dim=1)
```

```

864     new_centroids = torch.zeros_like(centroids)
865     counts = torch.zeros(K, device=device, dtype=DTYPE)
866     new_centroids.scatter_add_(0, clusters[:, None].expand(-1, D), A)
867     counts.scatter_add_(0, clusters, torch.ones_like(clusters, dtype=
868         DTYPE))
869     mask = counts > 0
870     new_centroids[mask] /= counts[mask, None]
871     empty_clusters = (counts == 0).nonzero(as_tuple=True)[0]
872     if empty_clusters.numel() > 0:
873         new_centroids[empty_clusters] = A[torch.randint(0, N, (
874             empty_clusters.numel(),))]
875     centroids = new_centroids
876     if torch.max(torch.norm(centroids - prev_centroids, dim=1)) < tol:
877         break
878     return clusters, centroids, iteration+1

```

LISTING 3: KMEANS + LOCAL KNN RETRIEVAL

Listing 3: ANN retrieval via KMeans clustering and local KNN

```

880
881
882 def our_ann(N:int, D:int, A:torch.Tensor, X:torch.Tensor, K:int, K_kmeans
883     :int=20, K_knn:int=10, distance_metric:str='dot', batch_size:int
884     =100000, max_iters:int=1000, tol:float=1e-4):
885     if K > N: raise ValueError("Invalid K, larger than N")
886     if distance_metric not in distance_types: raise ValueError(f"Invalid
887         distance metric: {distance_metric}.")
888     A = torch.as_tensor(A, dtype=DTYPE, device='cuda')
889     device = A.device
890     X = torch.as_tensor(X, dtype=DTYPE, device=device)
891     clusters, centroids, _ = our_kmeans(N, D, A, K_kmeans, distance_metric
892         , max_iters, tol)
893     top_indices, _ = our_knn(K_kmeans, D, centroids, X, K_knn,
894         distance_metric, batch_size)
895     candidate_indices = torch.cat([torch.nonzero(clusters == idx, as_tuple
896         =True)[0] for idx in top_indices])
897     if len(candidate_indices) < K:
898         warnings.warn(f"Only {len(candidate_indices)} candidates found.",
899             UserWarning)
900         K = len(candidate_indices)
901         candidates = A[candidate_indices]
902         top_k_indices, top_k_distances = our_knn(len(candidate_indices), D,
903             candidates, X, K, distance_metric, batch_size)
904         top_k_indices = candidate_indices[top_k_indices]
905     return top_k_indices, top_k_distances

```

These implementations reflect the actual retrieval code used in our experiments and benchmarking results in Sections 1.4 and 1.5.

G IVFPQ BENCHMARK COMPARISON

We compared our KMeans+KNN ANN approach with FAISS-IVFPQ at both small and large scales. The following tables report recall and average runtime.

Small Dataset (N=1000, D=100):

Distance	Recall	Time (ms)	IVFPQ Configs
Cosine	0.30–0.40	724–1447	Kivf=32/64, Kprobe=4/16, Kpq=5/10
L2	0.10–0.40	722–1430	same
Dot	0.20	712–1265	same
L1	0.20–0.50	725–1431	same

Table 4: IVFPQ results for small dataset.

Large Dataset (N=200,000, D=100):

Distance	Recall	Time (ms)	Config
Cosine	0.12–0.29	3125–4130	varied
L2	0.07–0.17	3012–3857	varied
Dot	0.10–0.25	3001–3827	varied
L1	0.07–0.17	3649–4633	varied

Table 5: IVFPQ results for large dataset.

Pipeline Comparison Summary (N=200,000, D=100):

Pipeline	Avg. Recall	Avg. Time (ms)	Notes
Exact Top-K (GPU)	1.00	22000–25000	Fully accurate but slowest
KMeans+KNN (Ours)	0.74–0.81	11000–16000	Good recall, 1.5–2× faster than exact
IVFPQ (FAISS)	0.10–0.29	3000–4300	Fastest but low recall

Table 6: Average recall and runtime across all methods on the large dataset.

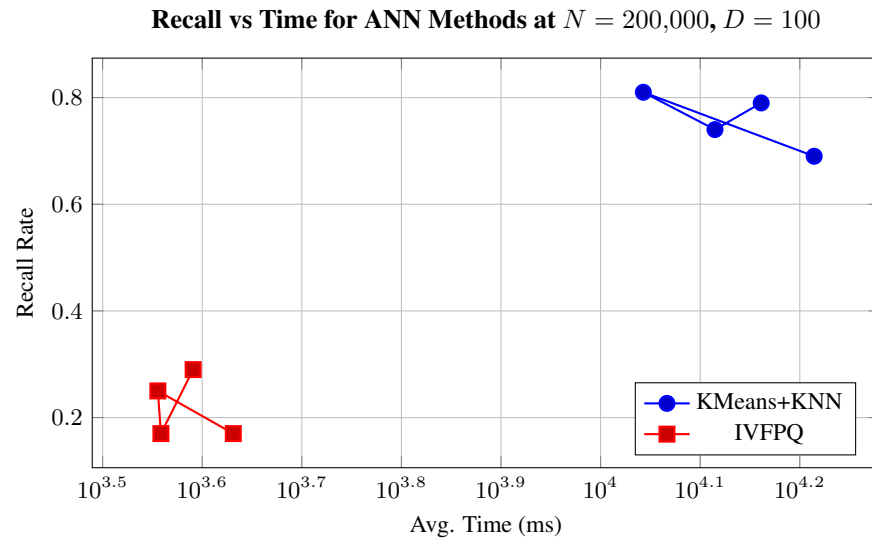


Figure 6: Comparison of average recall vs. execution time for FAISS-IVFPQ and our custom ANN (KMeans+KNN) implementation. FAISS is faster but less accurate, while our method maintains higher recall.

H IVFPQ IMPLEMENTATION

This appendix presents the complete code implementation of our IVFPQ-based approximate nearest neighbor (ANN) pipeline. The code performs inverted file indexing with product quantization and runs entirely on GPU using PyTorch. This function was used to generate the benchmarks in Section 1.5 and Appendix G.

LISTING 4: IVFPQ PIPELINE WITH COARSE CLUSTERING AND PQ LOOKUP

Listing 4: IVFPQ-based ANN retrieval with KMeans and Product Quantization

```
def our_ivfpq(N:int, D:int, A:torch.Tensor, X:torch.Tensor, K:int,
             K_ivf:int=100, K_probe:int=10, K_pq:int=4,
             distance_metric:str='dot', max_iters:int=5, pq_iters:int=3):
    if distance_metric not in distance_types:
        raise ValueError(f"Invalid distance metric: {distance_metric}")
    dist_multidim_func = dist_multidim_functions[distance_metric]
    assert D % K_pq == 0, "D must be divisible by K_pq"
    sub_dim = D // K_pq

    A = torch.as_tensor(A, dtype=DTYPE, device='cuda')
    device = A.device
    X = torch.as_tensor(X, dtype=DTYPE, device=device)

    # 1. Coarse clustering (IVF)
    centroids = A[torch.randperm(N)[:K_ivf]].clone()
    for _ in range(max_iters):
        dists = dist_multidim_func(A.unsqueeze(1), centroids.unsqueeze(0))
        assignments = dists.argmax(dim=1)
        for i in range(K_ivf):
            mask = assignments == i
            if mask.any():
                centroids[i] = A[mask].mean(dim=0)
    coarse_labels = assignments

    # 2. Product Quantization (PQ)
    pq_codebooks, pq_codes = [], []
    for i in range(K_pq):
        subvecs = A[:, i*sub_dim:(i+1)*sub_dim]
        codebook = subvecs[torch.randperm(N)[:256]].clone()
        for _ in range(pq_iters):
            dists = dist_multidim_func(subvecs.unsqueeze(1), codebook.
                                      unsqueeze(0))
            labels = dists.argmax(dim=1)
            for j in range(256):
                cluster = subvecs[labels == j]
                if cluster.shape[0] > 0:
                    codebook[j] = cluster.mean(dim=0)
        pq_codebooks.append(codebook)
        final_dists = dist_multidim_func(subvecs.unsqueeze(1), codebook.
                                       unsqueeze(0))
        pq_codes.append(final_dists.argmax(dim=1))
    pq_codes = torch.stack(pq_codes, dim=1)

    # 3. PQ Lookup Table
    pq_lut = torch.stack([
        dist_multidim_func(X[i*sub_dim:(i+1)*sub_dim].unsqueeze(0),
                          codebook).squeeze(0)
        for i, codebook in enumerate(pq_codebooks)
    ])

    # 4. Probe top clusters
    dists_q = dist_multidim_func(X.unsqueeze(0), centroids).squeeze(0)
    probe_ids = dists_q.topk(K_probe, largest=False).indices
```



```
1080
1081     # 5. Candidate selection and distance computation (ADC)
1082     candidate_indices = torch.cat([
1083         torch.nonzero(coarse_labels == idx, as_tuple=True)[0] for idx in
1084         probe_ids
1085     ], dim=0)
1086     if candidate_indices.numel() == 0:
1087         return torch.empty(0, device=device, dtype=torch.long), torch.empty
1088         (0, device=device)
1089     candidate_codes = pq_codes[candidate_indices]
1090     adc_dists = pq_lut[torch.arange(K_pq).unsqueeze(1), candidate_codes.T
1091         ].sum(dim=0)
1092
1093     # 6. Return Top-K
1094     top_k = min(K, candidate_indices.size(0))
1095     top_k_distances, top_k_idx = torch.topk(adc_dists, top_k, largest=
1096         False)
1097     top_k_indices = candidate_indices[top_k_idx]
1098     return top_k_indices, top_k_distances
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
```

I QUEUER AND BATCHER EXPERIMENTS RESULTS

Load (requests/s)	0.10	0.25	0.50	1	2	5	10	20	50	100
CPU-run	337.13	564.56	533.99	608.72	522.43	588.71	540.46	462.42	620.68	583.94
light	2.67	3.02	35.79	78.82	89.27	107.21	90.65	128.84	119.93	117.36
baseline	7.96	81.63	153.90	170.31	146.04	159.85	137.18	169.78	182.43	203.35
our_knn-based	8.17	66.62	128.64	127.30	187.18	157.92	185.27	171.56	165.64	162.53
queue-enriched 4-2	17.07	48.81	80.66	85.34	91.52	106.39	95.80	113.31	100.27	92.06
queue-enriched 8-4	26.81	75.27	114.03	94.88	111.85	109.70	115.42	116.81	100.24	112.19
queue-enriched 16-4	20.27	82.69	117.02	127.80	129.73	149.16	141.33	152.91	171.01	159.76

Table 7: Average response time (s) for standard requests set, testing RAG services at different loads. ('light' stands for the swallower opt-125m text-generator being used instead of Qwen2.5-1.5B-Instruct)
(integers couples characterize queue-enriched models by max batch size and waiting time respectively)